

Chương 2

Linear Algebra (Đại số tuyến tính)

Linear algebra (Đại số tuyến tính) là một nhánh của toán học được sử dụng rộng rãi trong toàn bộ khoa học và kỹ thuật. Tuy nhiên, vì linear algebra (đại số tuyến tính) là một dạng toán học liên tục (continuous) thay vì toán học rời rạc (discrete), nhiều nhà khoa học máy tính có ít kinh nghiệm với nó. Một sự hiểu biết tốt về linear algebra (đại số tuyến tính) là cần thiết để hiểu và làm việc với nhiều thuật toán machine learning (học máy), đặc biệt là các thuật toán deep learning (học sâu). Do đó, chúng tôi đặt trước phần giới thiệu về deep learning (học sâu) của mình một phần trình bày tập trung vào các kiến thức tiên quyết (prerequisites) quan trọng về linear algebra (đại số tuyến tính).

Nếu bạn đã quen thuộc với linear algebra (đại số tuyến tính), hãy thoải mái bỏ qua chương này. Nếu bạn đã có kinh nghiệm trước đó với các khái niệm này nhưng cần một bảng tham khảo chi tiết để xem lại các công thức chính, chúng tôi khuyên dùng *The Matrix Cookbook* (Petersen và Pedersen, 2006). Nếu bạn chưa từng tiếp xúc với linear algebra (đại số tuyến tính), chương này sẽ dạy cho bạn đủ để đọc cuốn sách này, nhưng chúng tôi thực sự khuyên bạn cũng nên tham khảo một tài liệu khác tập trung hoàn toàn vào việc dạy linear algebra (đại số tuyến tính), chẳng hạn như Shilov (1977). Chương này sẽ bỏ qua hoàn toàn nhiều chủ đề quan trọng của linear algebra (đại số tuyến tính) mà không cần thiết cho việc hiểu deep learning (học sâu).

2.1 Scalars, Vectors, Matrices và Tensors (Vô hướng, Vectơ, Ma trận và Tenxơ)

Việc nghiên cứu linear algebra (đại số tuyến tính) liên quan đến một vài loại đối tượng toán học:

- **Scalars (Vô hướng):** Một scalar (vô hướng) chỉ là một số đơn lẻ, trái ngược với hầu hết các đối tượng khác được nghiên cứu trong linear algebra (đại số tuyến tính), thường là các mảng (arrays) gồm nhiều số. Chúng tôi viết các scalars (vô hướng) bằng chữ in nghiêng. Chúng tôi thường đặt tên biến cho các scalars (vô hướng) bằng chữ cái in thường. Khi chúng tôi giới thiệu chúng, chúng tôi xác định rõ chúng là loại số nào. Ví dụ, chúng ta có thể nói "Cho $s \in \mathbb{R}$ là độ dốc của đường thẳng," khi định nghĩa một scalar (vô hướng) nhận giá trị thực, hoặc "Cho $n \in \mathbb{N}$ là số lượng đơn vị," khi định nghĩa một scalar (vô hướng) là số tự nhiên.
- **Vectors (Vectơ):** Một vector (vectơ) là một mảng (array) các số. Các số này được sắp xếp theo thứ tự. Chúng ta có thể xác định mỗi số riêng biệt bằng chỉ số (index) của nó trong thứ tự đó. Thông thường chúng tôi đặt tên cho các vectors (vectơ) bằng chữ cái in thường được viết in đậm, chẳng hạn như $\mathbf{x}$. Các phần tử (elements) của vector (vectơ) được xác định bằng cách viết tên của nó bằng chữ in nghiêng, với một chỉ số dưới (subscript). Phần tử đầu tiên của $\mathbf{x}$ là x_1 , phần tử thứ hai là x_2 và cứ tiếp tục như vậy. Chúng ta cũng cần cho biết loại số nào được lưu trữ trong vector (vectơ). Nếu mỗi phần tử thuộc $\mathbb{R}$, và vector (vectơ) có n phần tử, thì vector (vectơ) đó nằm trong tập hợp được tạo thành bằng cách lấy tích Descartes (Cartesian product) của $\mathbb{R}$ n lần, ký hiệu là $\mathbb{R}^n$. Khi chúng ta cần xác định rõ các

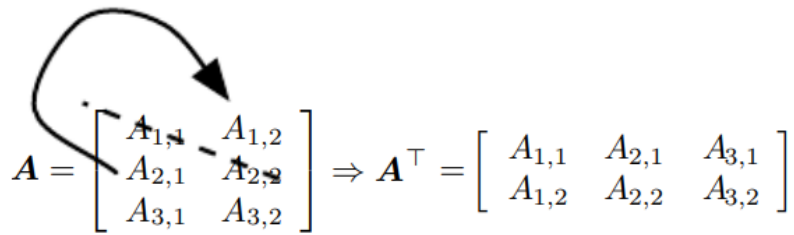
phần tử của một vector (vectơ), chúng ta viết chúng thành một cột nằm trong dấu ngoặc vuông:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}. \quad (1)$$

Chúng ta có thể nghĩ về các vectors (vectơ) như là việc xác định các điểm trong không gian, với mỗi phần tử cung cấp tọa độ dọc theo một trục (axis) khác nhau.

Đôi khi chúng ta cần lập chỉ mục (index) cho một tập hợp các phần tử của một vector (vectơ). Trong trường hợp này, chúng ta định nghĩa một tập hợp chứa các chỉ số và viết tập hợp đó ở dạng chỉ số dưới (subscript). Ví dụ, để truy cập x_1 , x_3 và x_6 , chúng ta định nghĩa tập hợp $S = \{1, 3, 6\}$ và viết $\mathbf{x}_S$. Chúng ta sử dụng dấu $-$ để lập chỉ mục phần bù (complement) của một tập hợp. Ví dụ $\mathbf{x}_{-1}$ là vector (vectơ) chứa tất cả các phần tử của $\mathbf{x}$ ngoại trừ x_1 , và $\mathbf{x}_{-S}$ là vector (vectơ) chứa tất cả các phần tử của $\mathbf{x}$ ngoại trừ x_1 , x_3 và x_6 .

- **Matrices (Ma trận):** Một matrix (ma trận) là một mảng 2 chiều (2-D array) các số, do đó mỗi phần tử được xác định bởi hai chỉ số (indices) thay vì chỉ một. Chúng tôi thường đặt tên biến cho các matrices (ma trận) bằng chữ cái in hoa, in đậm, chẳng hạn như $\mathbf{A}$. Nếu một matrix (ma trận) nhận giá trị thực $\mathbf{A}$ có chiều cao là m và chiều rộng là n , thì chúng ta nói rằng $\mathbf{A} \in \mathbb{R}^{m \times n}$. Chúng tôi thường xác định các phần tử của một matrix (ma trận) bằng cách sử dụng tên của nó in nghiêng nhưng không in đậm, và các chỉ số được liệt kê ngăn cách bằng dấu phẩy. Ví dụ, $A_{1,1}$ là phần tử ở trên cùng bên trái của $\mathbf{A}$ và $A_{m,n}$ là phần tử ở dưới cùng bên phải. Chúng ta có thể xác định tất cả các số có tọa độ dọc (vertical coordinate) là i bằng cách viết dấu ":" cho tọa độ ngang (horizontal coordinate). Ví dụ, $\mathbf{A}_{i,:}$ biểu thị mặt cắt ngang (horizontal cross section) của $\mathbf{A}$ với tọa độ dọc là i . Đây được gọi là hàng (row) thứ i của $\mathbf{A}$. Tương tự, $\mathbf{A}_{:,j}$ là...



$$\mathbf{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} \Rightarrow \mathbf{A}^\top = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \end{bmatrix}$$

Hình 1: Hình 2.1: Chuyển vị của ma trận có thể được coi là hình ảnh phản chiếu qua đường chéo chính.

... là cột thứ i của $\mathbf{A}$. Khi chúng ta cần xác định rõ ràng các phần tử của một ma trận, chúng ta viết chúng dưới dạng một mảng được đặt trong dấu ngoặc vuông:

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}. \quad (2)$$

Đôi khi chúng ta có thể cần lập chỉ mục cho các biểu thức có giá trị ma trận không chỉ là một chữ cái đơn lẻ. Trong trường hợp này, chúng ta sử dụng chỉ số dưới sau biểu thức, nhưng không chuyển đổi bất cứ thứ gì thành chữ in thường. Ví dụ, $f(\mathbf{A})_{i,j}$ cho phần tử (i, j) của matrix (ma trận) được tính bằng cách áp dụng hàm f cho $\mathbf{A}$.

- **Tensors (Tenxơ):** Trong một số trường hợp, chúng ta sẽ cần một mảng có nhiều hơn hai trục (axes). Trong trường hợp tổng quát, một mảng các số được sắp xếp trên một lưới đều với số lượng trục thay đổi được gọi là tensor (tenxơ). Chúng tôi ký hiệu một tensor (tenxơ) có tên "A" với kiểu chữ này: $\mathbf{A}$. Chúng ta xác định phần tử của $\mathbf{A}$ tại tọa độ (i, j, k) bằng cách viết $\mathbf{A}_{i,j,k}$.

Một phép toán quan trọng trên matrices (ma trận) là **transpose** (chuyển vị). **Transpose** (Chuyển vị) của một matrix (ma trận) là hình ảnh phản chiếu của matrix (ma trận) qua một đường chéo, được gọi là **main diagonal** (đường chéo chính), chạy xuống và sang phải, bắt đầu từ góc trên bên trái của nó. Xem hình 2.1 để biết mô tả đồ họa của phép toán này. Chúng tôi ký hiệu **transpose** (chuyển vị) của một matrix (ma trận) $\mathbf{A}$ là $\mathbf{A}^\top$, và nó được định nghĩa sao cho:

$$(\mathbf{A}^\top)_{i,j} = A_{j,i}. \quad (3)$$

Vectors (Vectơ) có thể được coi là matrices (ma trận) chỉ chứa một column (cột). Do đó, **transpose** (chuyển vị) của một vector (vectơ) là một matrix (ma trận) chỉ có một row (hàng). Đôi khi chúng ta định nghĩa một vector (vectơ) bằng cách viết các phần tử của nó trong văn bản nội tuyến (inline) dưới dạng row matrix (ma trận hàng), sau đó sử dụng toán tử **transpose** (chuyển vị) để biến nó thành standard column vector (vectơ cột tiêu chuẩn), ví dụ: $\mathbf{x} = [x_1, x_2, x_3]^\top$.

Một scalar (vô hướng) có thể được coi là một matrix (ma trận) chỉ có một phần tử duy nhất. Từ điều này, chúng ta có thể thấy rằng một scalar (vô hướng) là **transpose** (chuyển vị) của chính nó: $a = a^\top$.

Chúng ta có thể cộng các matrices (ma trận) với nhau, miễn là chúng có cùng shape (hình dạng), chỉ bằng cách cộng các phần tử tương ứng của chúng: $\mathbf{C} = \mathbf{A} + \mathbf{B}$ trong đó $C_{i,j} = A_{i,j} + B_{i,j}$.

Chúng ta cũng có thể cộng một scalar (vô hướng) vào một matrix (ma trận) hoặc nhân một matrix (ma trận) với một scalar (vô hướng), chỉ bằng cách thực hiện phép toán đó trên mỗi phần tử của matrix (ma trận): $\mathbf{D} = a \cdot \mathbf{B} + c$ trong đó $D_{i,j} = a \cdot B_{i,j} + c$.

Trong bối cảnh của deep learning (học sâu), chúng ta cũng sử dụng một số ký hiệu ít thông thường hơn. Chúng tôi cho phép cộng matrix (ma trận) và một vector (vectơ), tạo ra một matrix (ma trận) khác: $\mathbf{C} = \mathbf{A} + \mathbf{b}$, trong đó $C_{i,j} = A_{i,j} + b_j$. Nói cách khác, vector (vectơ) $\mathbf{b}$ được cộng vào mỗi row (hàng) của matrix (ma trận). Cách viết tắt này loại bỏ nhu cầu định nghĩa một matrix (ma trận) với $\mathbf{b}$ được sao chép vào mỗi hàng trước khi thực hiện phép cộng. Việc sao chép ngầm $\mathbf{b}$ đến nhiều vị trí này được gọi là **broadcasting** (phát sóng/lan truyền).

2.2 Multiplying Matrices and Vectors (Nhân Ma trận và Vectơ)

Một trong những phép toán quan trọng nhất liên quan đến matrices (ma trận) là phép nhân hai matrices (ma trận). **Matrix product** (Tích ma trận) của các matrices (ma trận) $\mathbf{A}$ và $\mathbf{B}$ là một matrix (ma trận) thứ ba $\mathbf{C}$. Để tích này được xác định, $\mathbf{A}$ phải có cùng số columns (cột) như $\mathbf{B}$ có số rows (hàng). Nếu $\mathbf{A}$ có shape (hình dạng) là $m \times n$

và $\mathbf{B}$ có kích thước là $n \times p$, thì $\mathbf{C}$ có kích thước là $m \times p$. Chúng ta có thể viết **matrix product** (tích ma trận) chỉ bằng cách đặt hai hoặc nhiều ma trận cạnh nhau, ví dụ:

$$\mathbf{C} = \mathbf{AB}. \quad (4)$$

Phép toán tích được định nghĩa bởi:

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}. \quad (5)$$

Lưu ý rằng tích tiêu chuẩn của hai matrices (ma trận) không chỉ là một matrix (ma trận) chứa tích của các phần tử riêng lẻ. Một phép toán như vậy tồn tại và được gọi là **element-wise product** (tích từng phần tử) hoặc **Hadamard product** (tích Hadamard), và được ký hiệu là $\mathbf{A} \odot \mathbf{B}$.

Dot product (Tích vô hướng) giữa hai vectors (vectơ) $\mathbf{x}$ và $\mathbf{y}$ có cùng số chiều là **matrix product** (tích ma trận) $\mathbf{x}^\top \mathbf{y}$. Chúng ta có thể coi **matrix product** (tích ma trận) $\mathbf{C} = \mathbf{AB}$ như việc tính toán $C_{i,j}$ là **dot product** (tích vô hướng) giữa row (hàng) i của $\mathbf{A}$ và column (cột) j của $\mathbf{B}$. Các phép toán **matrix product** (tích ma trận) có nhiều thuộc tính hữu ích giúp việc phân tích toán học của các **matrices** (ma trận) thuận tiện hơn. Ví dụ, phép nhân ma trận có tính **distributive** (phân phối):

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}. \quad (2.6)$$

Nó cũng có tính **associative** (kết hợp):

$$\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}. \quad (2.7)$$

Phép nhân ma trận *không* có tính **commutative** (giao hoán) (điều kiện $\mathbf{AB} = \mathbf{BA}$ không phải lúc nào cũng đúng), không giống như **scalar multiplication** (phép nhân vô hướng). Tuy nhiên, **dot product** (tích vô hướng) giữa hai **vectors** (vectơ) có tính **commutative** (giao hoán):

$$\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x}. \quad (2.8)$$

Transpose (Chuyển vị) của một **matrix product** (tích ma trận) có một dạng đơn giản:

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top. \quad (2.9)$$

Điều này cho phép chúng ta chứng minh phương trình 2.8, bằng cách khai thác thực tế là giá trị của một tích như vậy là một **scalar** (vô hướng) và do đó bằng chính **transpose** (chuyển vị) của nó:

$$\mathbf{x}^\top \mathbf{y} = (\mathbf{x}^\top \mathbf{y})^\top = \mathbf{y}^\top \mathbf{x}. \quad (2.10)$$

Vì trọng tâm của cuốn sách giáo khoa này không phải là **linear algebra** (đại số tuyến tính), chúng tôi không cố gắng phát triển một danh sách toàn diện các thuộc tính hữu ích của **matrix product** (tích ma trận) ở đây, nhưng người đọc nên lưu ý rằng còn nhiều thuộc tính khác tồn tại.

Bây giờ chúng ta đã biết đủ các ký hiệu **linear algebra** (đại số tuyến tính) để viết xuống một hệ phương trình tuyến tính (system of linear equations):

$$\mathbf{Ax} = \mathbf{b} \quad (2.11)$$

trong đó $\mathbf{A} \in \mathbb{R}^{m \times n}$ là một **matrix** (ma trận) đã biết, $\mathbf{b} \in \mathbb{R}^m$ là một **vector** (vectơ) đã biết, và $\mathbf{x} \in \mathbb{R}^n$ là một **vector** (vectơ) gồm các **unknown variables** (biến chưa biết) mà

chúng ta muốn giải. Mỗi phần tử x_i của $\mathbf{x}$ là một trong những biến chưa biết này. Mỗi **row** (hàng) của $\mathbf{A}$ và mỗi phần tử của $\mathbf{b}$ cung cấp một **constraint** (ràng buộc) khác. Chúng ta có thể viết lại phương trình 2.11 thành:

$$\mathbf{A}_{1,:}\mathbf{x} = b_1 \quad (2.12)$$

$$\mathbf{A}_{2,:}\mathbf{x} = b_2 \quad (2.13)$$

$$\dots \quad (2.14)$$

$$\mathbf{A}_{m,:}\mathbf{x} = b_m \quad (2.15)$$

hoặc, thậm chí rõ ràng hơn, thành:

$$A_{1,1}x_1 + A_{1,2}x_2 + \dots + A_{1,n}x_n = b_1 \quad (2.16)$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Hình 2: Figure 2.2: **Example identity matrix** (Ví dụ về ma trận đơn vị): Đây là $\mathbf{I}_3$.

$$A_{2,1}x_1 + A_{2,2}x_2 + \dots + A_{2,n}x_n = b_2 \quad (2.17)$$

$$\dots \quad (2.18)$$

$$A_{m,1}x_1 + A_{m,2}x_2 + \dots + A_{m,n}x_n = b_m. \quad (2.19)$$

Ký hiệu **matrix-vector product** (tích ma trận-vectơ) cung cấp một **representation** (biểu diễn) nhỏ gọn hơn cho các phương trình dạng này.

2.3 Identity and Inverse Matrices (Ma trận Đơn vị và Ma trận Nghịch đảo)

Linear algebra (Đại số tuyến tính) cung cấp một công cụ mạnh mẽ được gọi là **matrix inversion** (nghịch đảo ma trận) cho phép chúng ta giải phương trình 2.11 một cách giải tích (analytically) cho nhiều giá trị của $\mathbf{A}$.

Để mô tả **matrix inversion** (nghịch đảo ma trận), trước tiên chúng ta cần định nghĩa khái niệm **identity matrix** (ma trận đơn vị). Một **identity matrix** (ma trận đơn vị) là một matrix (ma trận) không làm thay đổi bất kỳ vector (vectơ) nào khi chúng ta nhân vector (vectơ) đó với matrix (ma trận) đó. Chúng tôi ký hiệu **identity matrix** (ma trận đơn vị) bảo toàn các vectors (vectơ) n -chiều (n-dimensional) là $\mathbf{I}_n$. Về mặt hình thức, $\mathbf{I}_n \in \mathbb{R}^{n \times n}$, và

$$\forall \mathbf{x} \in \mathbb{R}^n, \mathbf{I}_n \mathbf{x} = \mathbf{x}. \quad (2.20)$$

Cấu trúc của **identity matrix** (ma trận đơn vị) rất đơn giản: tất cả các phần tử dọc theo **main diagonal** (đường chéo chính) đều là 1, trong khi tất cả các phần tử khác đều là 0. Xem hình 2.2 cho một ví dụ.

Matrix inverse (Ma trận nghịch đảo) của $\mathbf{A}$ được ký hiệu là $\mathbf{A}^{-1}$, và nó được định nghĩa là matrix (ma trận) sao cho

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n. \quad (2.21)$$

Bây giờ chúng ta có thể giải phương trình 2.11 bằng các bước sau:

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad (2.22)$$

$$\mathbf{A}^{-1}\mathbf{A}\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} \quad (2.23)$$

$$\mathbf{I}_n\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} \quad (2.24)$$

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}. \quad (2.25)$$

Tất nhiên, quá trình này phụ thuộc vào việc có thể tìm được ma trận nghịch đảo $\mathbf{A}^{-1}$ hay không.

Về mặt lý thuyết, $\mathbf{A}^{-1}$ rất hữu ích. Tuy nhiên, trong thực tế lập trình phần mềm, ma trận nghịch đảo hiếm khi được sử dụng trực tiếp do các giới hạn về độ chính xác biểu diễn số thực trên máy tính. Các thuật toán giải hệ phương trình trực tiếp từ giá trị của $\mathbf{b}$ thường mang lại kết quả ước lượng $\mathbf{x}$ chính xác hơn.

2.4 Sự phụ thuộc tuyến tính và Không gian sinh

Để $\mathbf{A}^{-1}$ tồn tại, phương trình 2.11 ($\mathbf{A}\mathbf{x} = \mathbf{b}$) phải có nghiệm duy nhất với mọi giá trị của $\mathbf{b}$. Một hệ phương trình tuyến tính chỉ có thể xảy ra một trong ba trường hợp: vô nghiệm, có đúng một nghiệm, hoặc vô số nghiệm. Không bao giờ có trường hợp hệ phương trình có một số lượng nghiệm hữu hạn (lớn hơn 1). Ví dụ, nếu $\mathbf{x}$ và $\mathbf{y}$ đều là nghiệm, thì mọi điểm nằm trên đường thẳng nối giữa chúng cũng là nghiệm:

$$\mathbf{z} = \alpha\mathbf{x} + (1 - \alpha)\mathbf{y} \quad (2.26)$$

với mọi số thực α .

Để phân tích số lượng nghiệm, ta có thể hình dung các cột của ma trận $\mathbf{A}$ như các hướng di chuyển khác nhau từ gốc tọa độ. Khi đó, mỗi phần tử x_i của vectơ $\mathbf{x}$ sẽ đóng vai trò là "quãng đường" ta di chuyển theo hướng của cột thứ i :

$$\mathbf{A}\mathbf{x} = \sum_i x_i \mathbf{A}_{:,i}. \quad (2.27)$$

Phép toán nhân mỗi vectơ với một số vô hướng rồi cộng chúng lại với nhau được gọi là **tổ hợp tuyến tính**:

$$\sum_i c_i \mathbf{v}^{(i)}. \quad (2.28)$$

Tập hợp tất cả các điểm có thể đạt được thông qua phép tổ hợp tuyến tính của một tập vectơ ban đầu được gọi là **không gian sinh** (span) của chúng.

Do đó, việc xác định xem hệ phương trình $\mathbf{A}\mathbf{x} = \mathbf{b}$ có nghiệm hay không tương đương với việc kiểm tra xem vectơ $\mathbf{b}$ có nằm trong không gian sinh của các cột của $\mathbf{A}$ hay không. Tập hợp không gian sinh đặc biệt này được gọi là **không gian cột** hoặc **tập giá trị** của ma trận $\mathbf{A}$.

Để hệ $\mathbf{Ax} = \mathbf{b}$ có nghiệm với mọi $\mathbf{b} \in \mathbb{R}^m$, không gian cột của $\mathbf{A}$ phải bao trùm toàn bộ $\mathbb{R}^m$. Điều này đòi hỏi ma trận $\mathbf{A}$ phải có ít nhất m cột (tức là $n \geq m$). Ví dụ, một ma trận 3×2 chỉ có thể tạo ra tối đa một mặt phẳng 2D trong không gian 3D, do đó phương trình chỉ có nghiệm nếu $\mathbf{b}$ nằm trên mặt phẳng đó.

Tuy nhiên, $n \geq m$ chỉ là điều kiện cần chứ chưa đủ, vì các cột có thể bị dư thừa. Sự dư thừa này được gọi là **phụ thuộc tuyến tính**. Một tập hợp các vectơ là **độc lập tuyến tính** nếu không có vectơ nào trong tập có thể biểu diễn dưới dạng tổ hợp tuyến tính của các vectơ còn lại. Do đó, để không gian cột bao phủ toàn bộ $\mathbb{R}^m$, ma trận $\mathbf{A}$ phải chứa đúng m cột độc lập tuyến tính.

Hơn nữa, để ma trận có nghịch đảo, hệ phương trình chỉ được phép có *tối đa* một nghiệm cho mỗi giá trị của $\mathbf{b}$ (để tránh việc có vô số nghiệm). Điều này đòi hỏi ma trận có tối đa m cột.

Kết hợp các điều kiện trên, để có ma trận nghịch đảo, ma trận đó phải là **ma trận vuông** ($m = n$) và tất cả các cột phải độc lập tuyến tính. Một ma trận vuông chứa các cột phụ thuộc tuyến tính được gọi là **ma trận suy biến**.

Nếu $\mathbf{A}$ không vuông hoặc bị suy biến, ta vẫn có thể giải phương trình, nhưng không thể dùng phương pháp nghịch đảo ma trận. Ở phần trước ta đã bàn về ma trận nghịch đảo nhân bên trái, ta cũng có thể định nghĩa nghịch đảo nhân bên phải:

$$\mathbf{AA}^{-1} = \mathbf{I}. \quad (2.29)$$

Đối với ma trận vuông, nghịch đảo trái và nghịch đảo phải là như nhau.

2.5 Norms (Chuẩn)

Trong học máy, ta thường đo độ lớn của một vectơ bằng một hàm gọi là **norms**. Norms L^p được định nghĩa là:

$$\|\mathbf{x}\|_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}} \quad (2.30)$$

với $p \in \mathbb{R}, p \geq 1$.

Chuẩn ánh xạ vectơ thành các giá trị không âm, thể hiện khoảng cách từ gốc tọa độ đến điểm $\mathbf{x}$. Về mặt toán học nghiêm ngặt, một hàm f là một chuẩn nếu thỏa mãn ba tính chất:

- $f(\mathbf{x}) = 0 \Rightarrow \mathbf{x} = \mathbf{0}$
- $f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$ (bất đẳng thức tam giác)
- $\forall \alpha \in \mathbb{R}, f(\alpha \mathbf{x}) = |\alpha|f(\mathbf{x})$

Norms L^2 ($p = 2$) chính là **Norms Euclid** (khoảng cách Euclid từ gốc). Nó rất phổ biến trong học máy và thường được viết gọn là $\|\mathbf{x}\|$. Người ta cũng hay dùng bình phương chuẩn L^2 , có thể tính đơn giản bằng $\mathbf{x}^\top \mathbf{x}$.

Bình phương chuẩn L^2 tiện lợi hơn trong toán học và lập trình vì đạo hàm của nó đối với mỗi phần tử x_i chỉ phụ thuộc vào chính phần tử đó, trong khi đạo hàm của chuẩn L^2 thông thường lại phụ thuộc vào toàn bộ vectơ. Tuy nhiên, nhược điểm của nó là tăng giá trị rất chậm khi ở gần gốc tọa độ. Trong nhiều ứng dụng học máy, việc phân biệt giữa

các phần tử bằng không và các phần tử rất nhỏ (nhưng khác không) là rất quan trọng. Khi đó, ta dùng **chuẩn** L^1 :

$$||\mathbf{x}||_1 = \sum_i |x_i|. \quad (2.31)$$

Chuẩn L^1 tăng trưởng tuyến tính: mỗi khi một phần tử dịch chuyển khỏi số 0 một lượng ϵ , chuẩn L^1 cũng tăng đúng một lượng ϵ .

Đôi khi người ta đếm số lượng phần tử khác không của vectơ để đo kích thước của nó và hay gọi nhầm là "chuẩn L^0 ". Thực chất, nó không phải là một chuẩn hợp lệ vì việc nhân vectơ với một hằng số α không làm thay đổi số lượng phần tử khác không. Trong thực tế, chuẩn L^1 thường được dùng làm giải pháp thay thế cho mục đích này.

Một chuẩn phổ biến khác trong học máy là chuẩn L^∞ , hay còn gọi là **chuẩn max**. Nó đơn giản là giá trị tuyệt đối lớn nhất trong số các phần tử của vectơ:

$$||\mathbf{x}||_\infty = \max_i |x_i|. \quad (2.32)$$

Để đo kích thước của một ma trận, ta thường sử dụng **chuẩn Frobenius**:

$$||\mathbf{A}||_F = \sqrt{\sum_{i,j} A_{i,j}^2} \quad (2.33)$$

Chuẩn này tương tự như chuẩn L^2 của một vectơ.

Tích vô hướng của hai vectơ có thể được viết lại dưới dạng chuẩn như sau:

$$\mathbf{x}^\top \mathbf{y} = ||\mathbf{x}||_2 ||\mathbf{y}||_2 \cos \theta \quad (2.34)$$

trong đó θ là góc giữa $\mathbf{x}$ và $\mathbf{y}$.

2.6 Các loại Ma trận và Vectơ Đặc biệt

Ma trận đường chéo là ma trận hầu như chỉ chứa các số 0, ngoại trừ các phần tử nằm trên đường chéo chính. Về mặt hình thức, ma trận $\mathbf{D}$ là ma trận đường chéo nếu và chỉ nếu $D_{i,j} = 0$ với mọi $i \neq j$. Ma trận đơn vị chính là một ví dụ điển hình.

Ta dùng ký hiệu $\text{diag}(\mathbf{v})$ để chỉ một ma trận đường chéo vuông có các phần tử trên đường chéo được lấy từ vectơ $\mathbf{v}$.

Làm việc với ma trận đường chéo mang lại hiệu quả tính toán rất cao. Để tính tích $\text{diag}(\mathbf{v})\mathbf{x}$, ta chỉ cần nhân từng phần tử tương ứng: $\text{diag}(\mathbf{v})\mathbf{x} = \mathbf{v} \odot \mathbf{x}$.

Việc tính ma trận nghịch đảo cũng rất dễ dàng. Nghịch đảo chỉ tồn tại khi mọi phần tử trên đường chéo đều khác 0, và khi đó:

$$\text{diag}(\mathbf{v})^{-1} = \text{diag}([1/v_1, \dots, 1/v_n]^\top) \quad (6)$$

Chính vì vậy, nhiều thuật toán học máy có thể giảm thiểu đáng kể chi phí tính toán bằng cách giới hạn một số ma trận ở dạng đường chéo. Ma trận đường chéo không nhất thiết phải là ma trận vuông (có thể là hình chữ nhật). Ma trận đường chéo không vuông không có nghịch đảo, nhưng phép nhân với chúng vẫn rất tiết kiệm chi phí tính toán. Đối với một ma trận đường chéo không vuông $\mathbf{D}$, tích $\mathbf{D}\mathbf{x}$ chỉ đơn giản là nhân vô hướng từng phần tử của $\mathbf{x}$, sau đó thêm các số 0 vào kết quả (nếu $\mathbf{D}$ cao hơn rộng) hoặc cắt bỏ một vài phần tử cuối của vectơ (nếu $\mathbf{D}$ rộng hơn cao).

Ma trận đối xứng là ma trận bằng chính ma trận chuyển vị của nó:

$$\mathbf{A} = \mathbf{A}^\top. \quad (2.35)$$

Ma trận đối xứng thường xuất hiện khi các phần tử được tạo ra bởi một hàm tính toán không phụ thuộc vào thứ tự tham số (ví dụ: ma trận khoảng cách, khoảng cách từ i đến j bằng từ j đến i).

Vectơ đơn vị là vectơ có chuẩn bằng 1:

$$\|\mathbf{x}\|_2 = 1. \quad (2.36)$$

Hai vectơ $\mathbf{x}$ và $\mathbf{y}$ được gọi là **trực giao** với nhau nếu $\mathbf{x}^\top \mathbf{y} = 0$. Nếu cả hai đều có chuẩn khác 0, điều này đồng nghĩa với việc chúng vuông góc với nhau (tạo góc 90 độ). Trong không gian $\mathbb{R}^n$, có tối đa n vectơ trực giao với nhau từng đôi một (và khác 0). Nếu các vectơ này không chỉ trực giao mà còn là vectơ đơn vị, ta gọi chúng là **trực chuẩn**.

Ma trận trực giao là một ma trận vuông có các hàng và các cột đều trực chuẩn với nhau từng đôi một:

$$\mathbf{A}^\top \mathbf{A} = \mathbf{A} \mathbf{A}^\top = \mathbf{I}. \quad (2.37)$$

Điều này dẫn đến một hệ quả rất hữu ích:

$$\mathbf{A}^{-1} = \mathbf{A}^\top, \quad (2.38)$$

Do đó, ma trận nghịch đảo của một ma trận trực giao cực kỳ dễ tính toán. Lưu ý: định nghĩa của ma trận trực giao yêu cầu các hàng/cột phải *trực chuẩn* chứ không chỉ đơn thuần là trực giao.

2.7 Phân tích giá trị riêng (Eigendecomposition)

Tương tự như việc ta có thể phân tích một số nguyên thành các thừa số nguyên tố (ví dụ: $12 = 2 \times 2 \times 3$) để hiểu rõ bản chất cấu trúc của nó, ta cũng có thể phân tích các ma trận thành các thành phần cơ bản để làm lộ ra những đặc tính ẩn sâu bên trong mà việc nhìn vào mảng số thông thường không thể thấy được.

Một trong những phương pháp phân tích phổ biến nhất là **phân tích giá trị riêng**, trong đó ta phân tích một ma trận thành một tập hợp các **vectơ riêng** và **giá trị riêng**.

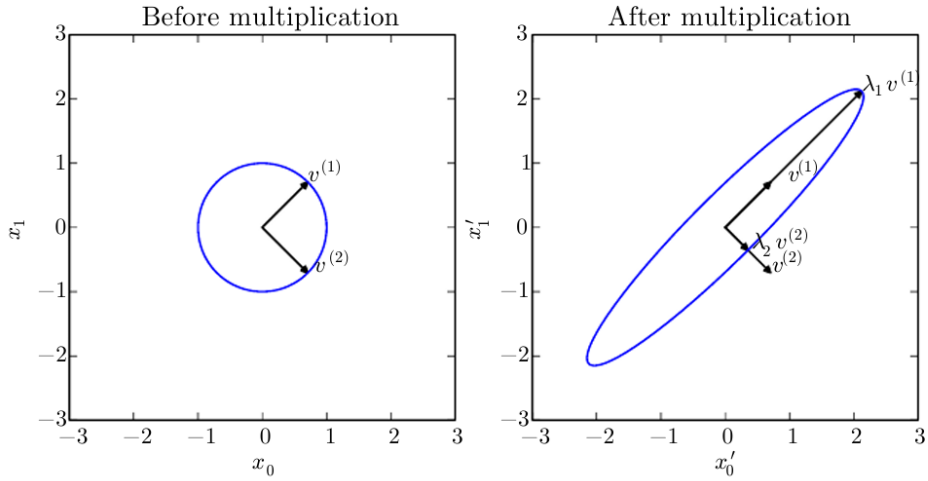
Vectơ riêng của một ma trận vuông $\mathbf{A}$ là một vectơ $\mathbf{v}$ khác vectơ không, sao cho khi nhân với $\mathbf{A}$, nó chỉ bị thay đổi về tỷ lệ (độ lớn) chứ không đổi hướng:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}. \quad (2.39)$$

Số vô hướng λ được gọi là **giá trị riêng** tương ứng với vectơ riêng này. (Người ta cũng có thể tìm thấy một **vectơ riêng trái** sao cho $\mathbf{v}^\top \mathbf{A} = \lambda \mathbf{v}^\top$, nhưng chúng ta thường chỉ quan tâm đến các vectơ riêng phải).

Nếu $\mathbf{v}$ là một vectơ riêng của $\mathbf{A}$, thì bất kỳ vectơ nào được thay đổi tỷ lệ $s\mathbf{v}$ với $s \in \mathbb{R}, s \neq 0$ cũng là một vectơ riêng. Hơn nữa, $s\mathbf{v}$ vẫn có cùng giá trị riêng. Vì lý do này, chúng ta thường chỉ tìm kiếm các vectơ riêng đơn vị.

Giả sử rằng một ma trận $\mathbf{A}$ có n vectơ riêng độc lập tuyến tính, $\{\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}\}$, với các giá trị riêng tương ứng $\{\lambda_1, \dots, \lambda_n\}$. Chúng ta có thể ghép tất cả các...



Hình 3: Một ví dụ về tác động của các vectơ riêng và giá trị riêng. Ở đây, chúng ta có một ma trận $\mathbf{A}$ với hai vectơ riêng trực chuẩn, $\mathbf{v}^{(1)}$ với giá trị riêng λ_1 và $\mathbf{v}^{(2)}$ với giá trị riêng λ_2 . (Trái) Chúng ta vẽ tập hợp tất cả các vectơ đơn vị $\mathbf{u} \in \mathbb{R}^2$ tạo thành một hình tròn đơn vị. (Phải) Chúng ta vẽ tập hợp tất cả các điểm $\mathbf{A}\mathbf{u}$. Bằng cách quan sát cách mà $\mathbf{A}$ làm biến dạng hình tròn đơn vị, chúng ta có thể thấy rằng nó kéo giãn không gian theo hướng $\mathbf{v}^{(i)}$ với tỷ lệ λ_i .

...vectơ riêng để tạo thành một ma trận $\mathbf{V}$ với mỗi cột là một vectơ riêng: $\mathbf{V} = [\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}]$. Tương tự, chúng ta có thể ghép các giá trị riêng để tạo thành một vectơ $\boldsymbol{\lambda} = [\lambda_1, \dots, \lambda_n]^\top$. **Phân tích giá trị riêng** của $\mathbf{A}$ khi đó được cho bởi:

$$\mathbf{A} = \mathbf{V} \text{diag}(\boldsymbol{\lambda}) \mathbf{V}^{-1}. \quad (2.40)$$

Chúng ta đã thấy rằng việc *xây dựng* các ma trận với các giá trị riêng và vectơ riêng cụ thể cho phép chúng ta kéo giãn không gian theo các hướng mong muốn. Tuy nhiên, chúng ta thường muốn **phân tích** các ma trận thành các giá trị riêng và vectơ riêng của chúng. Làm như vậy có thể giúp chúng ta phân tích các thuộc tính nhất định của ma trận, giống như việc phân tích một số nguyên thành các thừa số nguyên tố của nó có thể giúp chúng ta hiểu được hành vi của số nguyên đó.

Không phải mọi ma trận đều có thể được phân tích thành các giá trị riêng và vectơ riêng. Trong một số trường hợp, phép phân tích này vẫn tồn tại nhưng lại chứa các số phức thay vì số thực. May mắn là trong cuốn sách này, ta chủ yếu làm việc với một lớp ma trận đặc biệt có phép phân tích rất đơn giản: Mọi ma trận đối xứng thực đều có thể phân tích hoàn toàn bằng các giá trị riêng và vectơ riêng thực:

$$\mathbf{A} = \mathbf{Q} \boldsymbol{\Lambda} \mathbf{Q}^\top, \quad (2.41)$$

trong đó $\mathbf{Q}$ là ma trận trực giao chứa các vectơ riêng của $\mathbf{A}$, và $\boldsymbol{\Lambda}$ là ma trận đường chéo chứa các giá trị riêng tương ứng.

Tóm tắt các tính chất quan trọng từ phép phân tích giá trị riêng:

- **Tính duy nhất:** Phép phân tích này sẽ là duy nhất nếu tất cả các giá trị riêng đều khác nhau (theo quy ước, ta thường sắp xếp các giá trị riêng trên $\boldsymbol{\Lambda}$ theo thứ tự giảm dần). Nếu có giá trị riêng lặp lại, phép phân tích có thể không duy nhất.

- **Tính suy biến:** Một ma trận bị suy biến khi và chỉ khi nó có ít nhất một giá trị riêng bằng 0.
- **Tối ưu hóa:** Khi cần tìm cực trị của hàm bậc hai $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ (với điều kiện $\|\mathbf{x}\|_2 = 1$), giá trị lớn nhất và nhỏ nhất của hàm này tương ứng chính là giá trị riêng lớn nhất và nhỏ nhất của ma trận $\mathbf{A}$.

Dựa vào dấu của các giá trị riêng, ta có thể phân loại ma trận:

- **Xác định dương (Positive definite):** Tất cả giá trị riêng > 0 . Điều này đảm bảo rằng với mọi $\mathbf{x} \neq \mathbf{0}$, ta luôn có $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$.
- **Nửa xác định dương (Positive semidefinite):** Tất cả giá trị riêng ≥ 0 . Điều này đảm bảo rằng với mọi $\mathbf{x}$, ta luôn có $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$.
- **Xác định âm / nửa xác định âm:** Tương tự như trên nhưng áp dụng khi các giá trị riêng < 0 hoặc ≤ 0 .

2.8 Phân tích giá trị suy biến (Singular Value Decomposition)

Ở phần 2.7, ta đã biết cách phân tích ma trận thành các vectơ riêng và giá trị riêng. **Phân tích giá trị suy biến (SVD)** cung cấp một phương pháp khác để phân tích ma trận thành các **vectơ suy biến (singular vectors)** và **giá trị suy biến (singular values)**. SVD cho phép ta trích xuất những thông tin tương tự như phân tích giá trị riêng. Tuy nhiên, SVD mang tính tổng quát cao hơn. Mọi ma trận thực đều có thể phân tích giá trị suy biến, trong khi điều này không đúng với phân tích giá trị riêng (ví dụ: ma trận không vuông không có phân tích giá trị riêng, bắt buộc phải dùng SVD).

Nhắc lại, phân tích giá trị riêng của ma trận $\mathbf{A}$ được viết dưới dạng:

$$\mathbf{A} = \mathbf{V} \text{diag}(\boldsymbol{\lambda}) \mathbf{V}^{-1}. \quad (2.42)$$

Tương tự, **phân tích giá trị suy biến (SVD)** phân tích ma trận $\mathbf{A}$ thành tích của ba ma trận:

$$\mathbf{A} = \mathbf{U} \mathbf{D} \mathbf{V}^\top. \quad (2.43)$$

Giả sử $\mathbf{A}$ là ma trận kích thước $m \times n$. Khi đó:

- $\mathbf{U}$ là ma trận trực giao kích thước $m \times m$. Các cột của $\mathbf{U}$ được gọi là các **vectơ suy biến trái**.
- $\mathbf{D}$ là ma trận đường chéo kích thước $m \times n$. Các phần tử trên đường chéo của $\mathbf{D}$ được gọi là các **giá trị suy biến**.
- $\mathbf{V}$ là ma trận trực giao kích thước $n \times n$. Các cột của $\mathbf{V}$ được gọi là các **vectơ suy biến phải**.

Mối liên hệ giữa SVD và phân tích giá trị riêng:

- Các vectơ suy biến trái của $\mathbf{A}$ chính là các vectơ riêng của ma trận $\mathbf{A} \mathbf{A}^\top$.
- Các vectơ suy biến phải của $\mathbf{A}$ chính là các vectơ riêng của ma trận $\mathbf{A}^\top \mathbf{A}$.

- Các giá trị suy biến khác không của $\mathbf{A}$ chính là căn bậc hai của các giá trị riêng của ma trận $\mathbf{A}^\top \mathbf{A}$ (và $\mathbf{A} \mathbf{A}^\top$).

Ứng dụng hữu ích nhất của SVD là giúp tổng quát hóa khái niệm ma trận nghịch đảo cho các ma trận không vuông.

2.9 Giả nghịch đảo Moore-Penrose

Phép nghịch đảo ma trận không áp dụng được cho ma trận không vuông. Giả sử ta muốn tìm một ma trận nghịch đảo trái $\mathbf{B}$ của $\mathbf{A}$ để giải hệ phương trình tuyến tính:

$$\mathbf{A}\mathbf{x} = \mathbf{y} \quad (2.44)$$

bằng cách nhân ma trận $\mathbf{B}$ vào bên trái cả hai vế để có được:

$$\mathbf{x} = \mathbf{B}\mathbf{y}. \quad (2.45)$$

Tùy thuộc vào cấu trúc của hệ, có thể không tồn tại một ánh xạ duy nhất từ $\mathbf{A}$ sang $\mathbf{B}$:

- Nếu $\mathbf{A}$ cao hơn rộng (nhiều phương trình hơn ẩn), hệ có thể vô nghiệm.
- Nếu $\mathbf{A}$ rộng hơn cao (nhiều ẩn hơn phương trình), hệ có thể có vô số nghiệm.

Giả nghịch đảo Moore-Penrose (Moore-Penrose pseudoinverse) cho phép ta giải quyết các trường hợp này. Giả nghịch đảo của ma trận $\mathbf{A}$, ký hiệu là $\mathbf{A}^+$, được định nghĩa thông qua giới hạn:

$$\mathbf{A}^+ = \lim_{\alpha \searrow 0} (\mathbf{A}^\top \mathbf{A} + \alpha \mathbf{I})^{-1} \mathbf{A}^\top. \quad (2.46)$$

Trong thực tế, các thuật toán tính ma trận giả nghịch đảo không dựa trên định nghĩa giới hạn, mà sử dụng công thức dựa trên phân tích SVD:

$$\mathbf{A}^+ = \mathbf{V} \mathbf{D}^+ \mathbf{U}^\top, \quad (2.47)$$

trong đó $\mathbf{U}$, $\mathbf{D}$ và $\mathbf{V}$ là các ma trận từ phân tích SVD của $\mathbf{A}$. Ma trận giả nghịch đảo $\mathbf{D}^+$ được tạo ra bằng cách lấy nghịch đảo các phần tử khác 0 trên đường chéo của $\mathbf{D}$, sau đó chuyển vị ma trận kết quả.

Dùng giả nghịch đảo để giải phương trình tuyến tính $\mathbf{A}\mathbf{x} = \mathbf{y}$ mang lại các nghiệm tối ưu tùy theo hình dạng của $\mathbf{A}$:

- Khi $\mathbf{A}$ có nhiều cột hơn hàng (hệ có vô số nghiệm): $\mathbf{x} = \mathbf{A}^+ \mathbf{y}$ sẽ cho ra nghiệm có **chuẩn Euclid** $\|\mathbf{x}\|_2$ **nhỏ nhất** trong số tất cả các nghiệm có thể.
- Khi $\mathbf{A}$ có nhiều hàng hơn cột (hệ có thể vô nghiệm): $\mathbf{x} = \mathbf{A}^+ \mathbf{y}$ sẽ cho ra nghiệm giúp **khoảng cách Euclid** $\|\mathbf{A}\mathbf{x} - \mathbf{y}\|_2$ **là nhỏ nhất** (nghiệm gần đúng nhất).

2.10 Toán tử Vết (Trace)

Toán tử vết tính tổng của tất cả các phần tử nằm trên đường chéo chính của một ma trận:

$$\text{Tr}(\mathbf{A}) = \sum_i A_{i,i}. \quad (2.48)$$

Toán tử vết rất hữu ích vì nó giúp viết lại một số phép toán phức tạp một cách gọn gàng. Ví dụ, chuẩn Frobenius có thể được viết lại thành:

$$\|\mathbf{A}\|_F = \sqrt{\text{Tr}(\mathbf{A}\mathbf{A}^\top)}. \quad (2.49)$$

Các tính chất quan trọng của Vết:

- Bất biến với phép chuyển vị:

$$\text{Tr}(\mathbf{A}) = \text{Tr}(\mathbf{A}^\top). \quad (2.50)$$

- Bất biến với phép hoán vị vòng quanh (cyclic permutation):

$$\text{Tr}(\mathbf{A}\mathbf{B}\mathbf{C}) = \text{Tr}(\mathbf{C}\mathbf{A}\mathbf{B}) = \text{Tr}(\mathbf{B}\mathbf{C}\mathbf{A}). \quad (2.51)$$

- Tính chất trên có thể tổng quát hóa cho tích của nhiều ma trận:

$$\text{Tr}\left(\prod_{i=1}^n \mathbf{F}^{(i)}\right) = \text{Tr}\left(\mathbf{F}^{(n)} \prod_{i=1}^{n-1} \mathbf{F}^{(i)}\right). \quad (2.52)$$

- Sự bất biến này vẫn đúng ngay cả khi ma trận kết quả có kích thước khác nhau. Ví dụ, nếu $\mathbf{A} \in \mathbb{R}^{m \times n}$ và $\mathbf{B} \in \mathbb{R}^{n \times m}$, ta vẫn có:

$$\text{Tr}(\mathbf{A}\mathbf{B}) = \text{Tr}(\mathbf{B}\mathbf{A}) \quad (2.53)$$

mặc dù $\mathbf{A}\mathbf{B} \in \mathbb{R}^{m \times m}$ và $\mathbf{B}\mathbf{A} \in \mathbb{R}^{n \times n}$.

- Vết của một số vô hướng chính là số vô hướng đó: $a = \text{Tr}(a)$.

2.11 Định thức (Determinant)

Định thức của một ma trận vuông $\mathbf{A}$, ký hiệu là $\det(\mathbf{A})$, là một hàm ánh xạ ma trận đó thành một số vô hướng thực.

Ý nghĩa cốt lõi của định thức:

- Xét về mặt đại số, định thức bằng **tích của tất cả các giá trị riêng** của ma trận đó.
- Xét về mặt hình học, giá trị tuyệt đối của định thức đo lường mức độ mà phép nhân với ma trận đó làm **giãn nở hay co rút không gian**.
- Nếu $\det(\mathbf{A}) = 0$, không gian bị co rút hoàn toàn theo ít nhất một chiều, khiến thể tích của nó bằng 0.
- Nếu $\det(\mathbf{A}) = 1$, phép biến đổi ma trận đó bảo toàn nguyên vẹn thể tích của không gian.

2.12 Ví dụ: Phân tích thành phần chính (PCA)

Phân tích thành phần chính (PCA) là một thuật toán học máy đơn giản có thể được suy ra chỉ bằng kiến thức đại số tuyến tính cơ bản. Mục tiêu của nó là nén dữ liệu có mất mát (giảm kích thước bộ nhớ lưu trữ nhưng cố gắng giữ lại độ chính xác cao nhất có thể).

Thiết lập bài toán:

- Giả sử ta có m điểm dữ liệu $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ trong không gian $\mathbb{R}^n$.
- Ta muốn nén chúng thành các vectơ mã $\mathbf{c}^{(i)} \in \mathbb{R}^l$ ở không gian có số chiều nhỏ hơn ($l < n$).
- Ta cần một hàm mã hóa $f(\mathbf{x}) = \mathbf{c}$ và một hàm giải mã $\mathbf{x} \approx g(\mathbf{c})$.

Cách tiếp cận của PCA: PCA định nghĩa hàm giải mã là một phép nhân ma trận cực kỳ đơn giản:

$$g(\mathbf{c}) = \mathbf{D}\mathbf{c} \quad (7)$$

trong đó $\mathbf{D} \in \mathbb{R}^{n \times l}$ là ma trận giải mã.

Để bài toán dễ giải và có nghiệm duy nhất, PCA ràng buộc các cột của $\mathbf{D}$ phải **trực giao** với nhau và là các **vectơ đơn vị** (có chuẩn bằng 1).

Tìm mã tối ưu $\mathbf{c}^*$: Để tìm mã $\mathbf{c}^*$ tối ưu cho mỗi điểm input $\mathbf{x}$, ta cần tối thiểu hóa khoảng cách giữa $\mathbf{x}$ và bản phục hồi của nó $g(\mathbf{c}^*)$ bằng chuẩn L^2 :

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} \|\mathbf{x} - g(\mathbf{c})\|_2 \quad (2.54)$$

Vì chuẩn L^2 luôn không âm, việc tối thiểu hóa nó tương đương với việc tối thiểu hóa bình phương chuẩn L^2 của nó:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} \|\mathbf{x} - g(\mathbf{c})\|_2^2 \quad (2.55)$$

Khai triển hàm cần tối thiểu hóa này, ta có:

$$(\mathbf{x} - g(\mathbf{c}))^\top (\mathbf{x} - g(\mathbf{c})) \quad (2.56)$$

(theo định nghĩa chuẩn L^2)

$$= \mathbf{x}^\top \mathbf{x} - \mathbf{x}^\top g(\mathbf{c}) - g(\mathbf{c})^\top \mathbf{x} + g(\mathbf{c})^\top g(\mathbf{c}) \quad (2.57)$$

(theo tính chất phân phối)

$$= \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top g(\mathbf{c}) + g(\mathbf{c})^\top g(\mathbf{c}) \quad (2.58)$$

(Bước 2.58 có được do $g(\mathbf{c})^\top \mathbf{x}$ là một số vô hướng nên nó bằng chính ma trận chuyển vị của nó).

Cuối cùng, ta có thể bỏ qua số hạng đầu tiên $\mathbf{x}^\top \mathbf{x}$ trong hàm mục tiêu vì nó chỉ phụ thuộc vào $\mathbf{x}$ chứ không phụ thuộc vào biến $\mathbf{c}$ mà ta đang cần tối ưu. Từ phần trước, sau khi bỏ qua số hạng không phụ thuộc vào $\mathbf{c}$, hàm mục tiêu trở thành:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} -2\mathbf{x}^\top g(\mathbf{c}) + g(\mathbf{c})^\top g(\mathbf{c}). \quad (2.59)$$

Thay hàm giải mã $g(\mathbf{c}) = \mathbf{D}\mathbf{c}$ vào phương trình:

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{D}^\top \mathbf{D}\mathbf{c} \quad (2.60)$$

$$= \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{I}_l \mathbf{c} \quad (2.61)$$

(do $\mathbf{D}$ có các cột trực giao và chuẩn đơn vị nên $\mathbf{D}^\top \mathbf{D} = \mathbf{I}_l$)

$$= \arg \min_{\mathbf{c}} -2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{c} \quad (2.62)$$

Ta giải bài toán tối ưu này bằng vi tích phân vectơ (lấy đạo hàm theo $\mathbf{c}$ và cho bằng $\mathbf{0}$):

$$\nabla_{\mathbf{c}}(-2\mathbf{x}^\top \mathbf{D}\mathbf{c} + \mathbf{c}^\top \mathbf{c}) = \mathbf{0} \quad (2.63)$$

$$-2\mathbf{D}^\top \mathbf{x} + 2\mathbf{c} = \mathbf{0} \quad (2.64)$$

$$\mathbf{c} = \mathbf{D}^\top \mathbf{x}. \quad (2.65)$$

Kết quả này cho thấy thuật toán cực kỳ hiệu quả: ta có thể mã hóa $\mathbf{x}$ chỉ bằng một phép nhân ma trận với vectơ. Hàm mã hóa được xác định là:

$$f(\mathbf{x}) = \mathbf{D}^\top \mathbf{x}. \quad (2.66)$$

Từ đó, phép khôi phục (giải mã) của PCA được định nghĩa là:

$$r(\mathbf{x}) = g(f(\mathbf{x})) = \mathbf{D}\mathbf{D}^\top \mathbf{x}. \quad (2.67)$$

Bước tiếp theo, ta cần tìm ma trận mã hóa $\mathbf{D}$. Lúc này, ta không xét từng điểm riêng lẻ nữa mà phải tối thiểu hóa chuẩn Frobenius của ma trận sai số trên **toàn bộ các điểm dữ liệu**:

$$\mathbf{D}^* = \arg \min_{\mathbf{D}} \sqrt{\sum_{i,j} \left(x_j^{(i)} - r(\mathbf{x}^{(i)})_j \right)^2} \quad \text{với điều kiện } \mathbf{D}^\top \mathbf{D} = \mathbf{I}_l \quad (2.68)$$

Để dễ hình dung, xét trường hợp $l = 1$ (chỉ nén xuống 1 chiều), ma trận $\mathbf{D}$ trở thành một vectơ duy nhất $\mathbf{d}$. Thay công thức 2.67 vào 2.68 và đơn giản hóa, bài toán trở thành:

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{d}\mathbf{d}^\top \mathbf{x}^{(i)}\|_2^2 \quad \text{với điều kiện } \|\mathbf{d}\|_2 = 1. \quad (2.69)$$

Cách viết trên dễ số vô hướng $\mathbf{d}^\top \mathbf{x}^{(i)}$ ở bên phải vectơ $\mathbf{d}$, trông không thuận mắt. Theo quy ước toán học, số vô hướng nên đặt bên trái vectơ, ta có thể viết lại phương trình trên thành:

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{d}^\top \mathbf{x}^{(i)} \mathbf{d}\|_2^2 \quad \text{với điều kiện } \|\mathbf{d}\|_2 = 1, \quad (2.70)$$

hoặc tận dụng tính chất số vô hướng bằng chính chuyển vị của nó ($\mathbf{d}^\top \mathbf{x}^{(i)} = \mathbf{x}^{(i)\top} \mathbf{d}$):

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \sum_i \|\mathbf{x}^{(i)} - \mathbf{x}^{(i)\top} \mathbf{d} \mathbf{d}\|_2^2 \quad \text{với điều kiện } \|\mathbf{d}\|_2 = 1. \quad (2.71)$$

(Người đọc nên làm quen với các phép biến đổi phương trình mang tính thẩm mỹ như thế này). Lúc này, thay vì viết dưới dạng tổng của các vectơ riêng lẻ, ta gộp tất cả các điểm dữ liệu vào một **ma trận thiết kế** (design matrix) $\mathbf{X} \in \mathbb{R}^{m \times n}$, sao cho mỗi hàng của $\mathbf{X}$ là một điểm dữ liệu: $\mathbf{X}_{i,:} = \mathbf{x}^{(i)\top}$.

Bài toán tối ưu được viết lại gọn gàng hơn dưới dạng chuẩn Frobenius:

$$\mathbf{d}^* = \arg \min_{\mathbf{d}} \|\mathbf{X} - \mathbf{X}\mathbf{d}\mathbf{d}^\top\|_F^2 \quad \text{với điều kiện } \mathbf{d}^\top \mathbf{d} = 1. \quad (2.72)$$

Tạm thời bỏ qua điều kiện $\mathbf{d}^\top \mathbf{d} = 1$, ta khai triển phần chuẩn Frobenius bằng toán tử Vết (Trace) theo phương trình 2.49 ($\|\mathbf{A}\|_F^2 = \text{Tr}(\mathbf{A}\mathbf{A}^\top)$):

$$\arg \min_{\mathbf{d}} \|\mathbf{X} - \mathbf{X}\mathbf{d}\mathbf{d}^\top\|_F^2 \quad (2.73)$$

$$= \arg \min_{\mathbf{d}} \text{Tr}((\mathbf{X} - \mathbf{X}\mathbf{d}\mathbf{d}^\top)^\top (\mathbf{X} - \mathbf{X}\mathbf{d}\mathbf{d}^\top)) \quad (2.74)$$

$$= \arg \min_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X} - \mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top - \mathbf{d}\mathbf{d}^\top \mathbf{X}^\top \mathbf{X} + \mathbf{d}\mathbf{d}^\top \mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) \quad (2.75)$$

Vì toán tử vết có tính chất tuyến tính, ta tách ra và loại bỏ các số hạng không chứa $\mathbf{d}$ (vì chúng không ảnh hưởng đến vị trí cực tiểu):

$$= \arg \min_{\mathbf{d}} -\text{Tr}(\mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) - \text{Tr}(\mathbf{d}\mathbf{d}^\top \mathbf{X}^\top \mathbf{X}) + \text{Tr}(\mathbf{d}\mathbf{d}^\top \mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) \quad (2.77)$$

Áp dụng tính chất hoán vị vòng quanh của Vết ($\text{Tr}(\mathbf{A}\mathbf{B}) = \text{Tr}(\mathbf{B}\mathbf{A})$), ta gộp hai số hạng đầu và viết lại số hạng cuối:

$$= \arg \min_{\mathbf{d}} -2\text{Tr}(\mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top \mathbf{d}\mathbf{d}^\top) \quad (2.79)$$

Đến đây, ta áp dụng lại điều kiện $\mathbf{d}^\top \mathbf{d} = 1$ vào số hạng cuối:

$$= \arg \min_{\mathbf{d}} -2\text{Tr}(\mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) + \text{Tr}(\mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) \quad (2.81)$$

$$= \arg \min_{\mathbf{d}} -\text{Tr}(\mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) \quad (2.82)$$

Việc tìm arg min của một hàm âm tương đương với việc tìm arg max của hàm dương:

$$= \arg \max_{\mathbf{d}} \text{Tr}(\mathbf{X}^\top \mathbf{X}\mathbf{d}\mathbf{d}^\top) \quad \text{với điều kiện } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.83)$$

Sử dụng lại tính chất hoán vị vòng quanh một lần nữa:

$$= \arg \max_{\mathbf{d}} \text{Tr}(\mathbf{d}^\top \mathbf{X}^\top \mathbf{X}\mathbf{d}) \quad \text{với điều kiện } \mathbf{d}^\top \mathbf{d} = 1 \quad (2.84)$$

Kết luận quan trọng của PCA: Dựa vào bài toán tối ưu trên, ta thấy rằng lời giải cho $\mathbf{d}$ chính là **vectơ riêng** của ma trận $\mathbf{X}^\top \mathbf{X}$ tương ứng với **giá trị riêng lớn nhất**.

Phần chứng minh trên áp dụng cho trường hợp nén xuống 1 chiều ($l = 1$). Để tổng quát hóa cho trường hợp muốn nén xuống không gian l chiều (khôi phục l thành phần chính), ma trận $\mathbf{D}$ sẽ được tạo thành từ l vectơ riêng tương ứng với l giá trị riêng lớn nhất của $\mathbf{X}^\top \mathbf{X}$. (Có thể chứng minh bằng quy nạp toán học).

Đại số tuyến tính là một trong những nền tảng toán học cốt lõi của Học sâu (Deep Learning). Một lĩnh vực toán học quan trọng không kém khác sẽ được trình bày ở chương tiếp theo chính là Lý thuyết Xác suất.